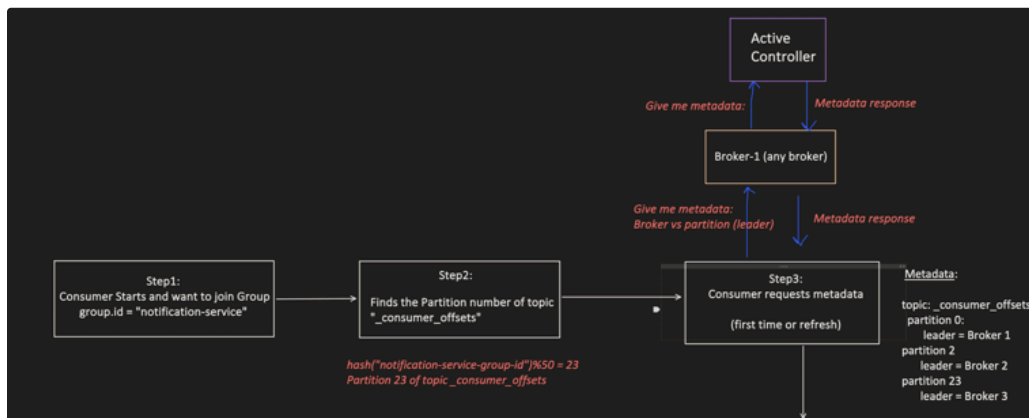


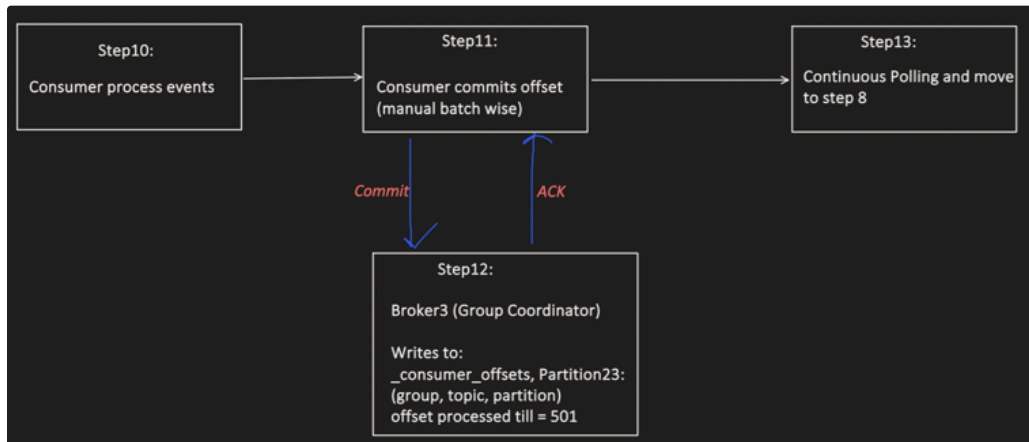
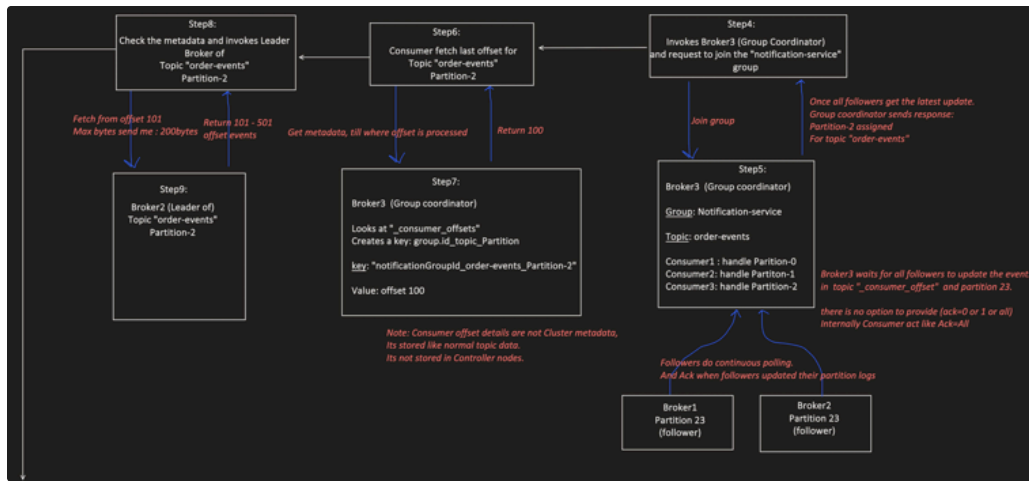
11. Consumer Setup - Part1

In Kafka Architecture topic, I have already covered the overall Consumer workflow:



Complete Consumer Read Flow:





Step1: Dependency.

pom.xml

```
1 <dependency>
2     <groupId>org.springframework.kafka</groupId>
3     <artifactId>spring-kafka</artifactId>
4 </dependency>
```

Step2: application.properties

```
1 server.port=8082
2 spring.application.name=kafka-consumer-service
3
4 spring.kafka.bootstrap-servers=localhost:9092,localhost:9192
5 <!--
6     Which group consumer needs to join
7 -->
8 spring.kafka.consumer.group-id=order-consumer-group
9 <!--
10     Should be same what at Producer we have selected
11 -->
12 spring.kafka.consumer.key-
    serializer=org.apache.kafka.common.serialization.StringSerializer
```

```

13 spring.kafka.consumer.value-
    serializer=org.springframework.kafka.support.serializer.JsonDeserial
    izer
14
15 <!--
16     Since we have selected JSON Deserializer , we need to tell the
    default type,
17     in which this JSON need to be converted to.
18     -->
19 spring.kafka.consumer.properties.spring.json.value.default.type=com.ed
    a.consumer.model.Order
20

```

POJO class

```

1 public class Order {
2     private String orderId;
3     private String customerId;
4     private String productId;
5     private Integer quantity;
6     private Double totalAmount;
7     private String status;
8
9     //getters and setters
10 }
11

```

Step3: Which topic need to listen and Business logic

```

1 @Component
2 public class OrderEventListener {
3     /*
4     Topic consumer interested in
5     */
6     @KafkaListener(topics="order-events")
7     public void consume (Order order) {
8         //business logic
9         /*
10        For each record(event), this method will get executed
11        */
12        System.out.println("Received order event: " +
    order.getOrderId());
13    }
14 }
15

```

DEMO

Now lets go 1 level deeper, what is happening behind the scene:

conceptually what we have to do without spring boot is:

```

1 Properties props = new Properties();
2     props.put("bootstrap.servers", "localhost:9092,localhost:9192");
3     <!--

```

```

4      Which group consumer needs to join
5      -->
6      props.put("group.id", "order-consumer-group");
7      <!--
8      Should be same what
9      producer has selected
10     -->
11     props.put("key.deserializer", StringDeserializer.class);
12     props.put("value.deserializer", JsonDeserializer.class);
13
14     props.put("auto.offset.reset", "latest");
15     props.put("enable.auto.commit", "true");
16
17     KafkaConsumer consumer = new KafkaConsumer<>(props);
18     <!--
19     Which topic, this consumer is interested in.
20     -->
21     consumer.subscribe(Arrays.asList("order-events"));
22
23     Connect with 1 of the broker and fetch cluster metadata and store it
24
25     try {
26         while (true) {
27             <!--
28             Continuous poll inside While(true) loop,
29             and it will fetch the records
30             -->
31             ConsumerRecords records = consumer.poll()
32             Internally poll() method, If it's a First
33             call:
34             w Sends JoinGroupRequest to Group
35             Coordinator
36             w Receives partition assignment
37             w Request last committed offset
38             for assigned partition from group coordinator
39             w Group coordinator reads from
40             _consumer_offsets and return the offset that need to be read.
41             w Sends FetchRequest to partition
42             leaders with offset which it want to read
43
44             If not a first calls:
45             w For subsequent calls, Sends
46             FetchRequest for new data (new offsets in continuous manner)
47
48             //commitIntervalPassed default is 5sec, after
49             every 5 second, make offset commit request
50             <!--
51             If auto-commit is enabled, then after specific time
52             interval, it commit the offset.
53             No matter if fetch poll records are successfully
54             processed or not.
55             -->
56             if (autoCommitEnabled && commitIntervalPassed) {
57                 commitOffsets();
58             }
59             <!--
60             Process the events
61             -->
62             for (ConsumerRecord record : records) {
63                 //business logic
64             }
65
66             If enable.auto.commit=false, we must call:
67             <!--
68             If auto-commit is false, then
69             Manually send the commit request
70             -->

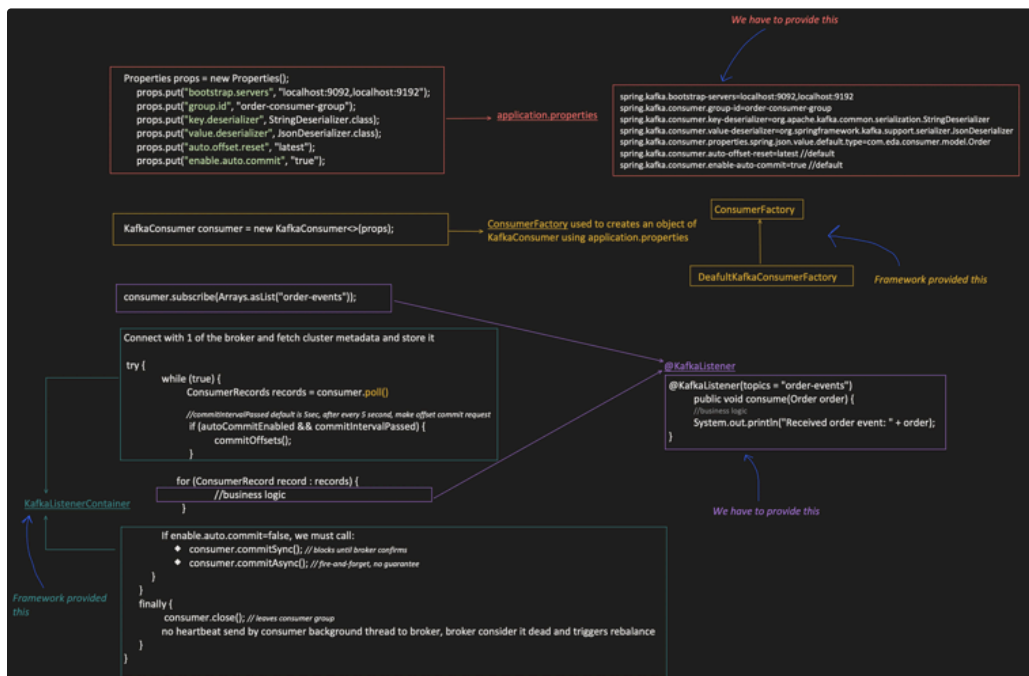
```

```

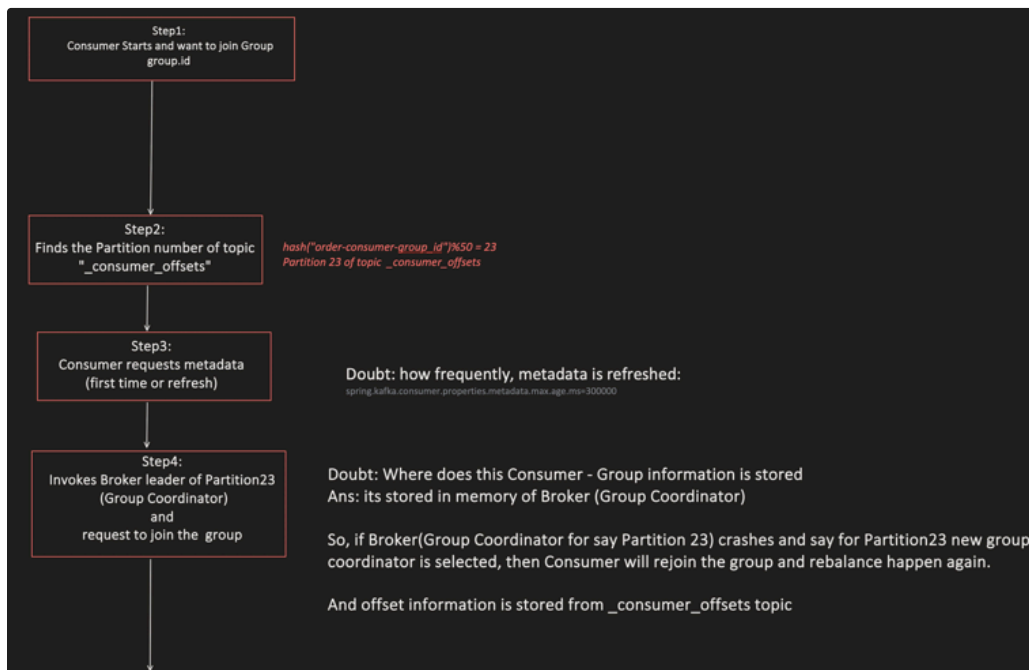
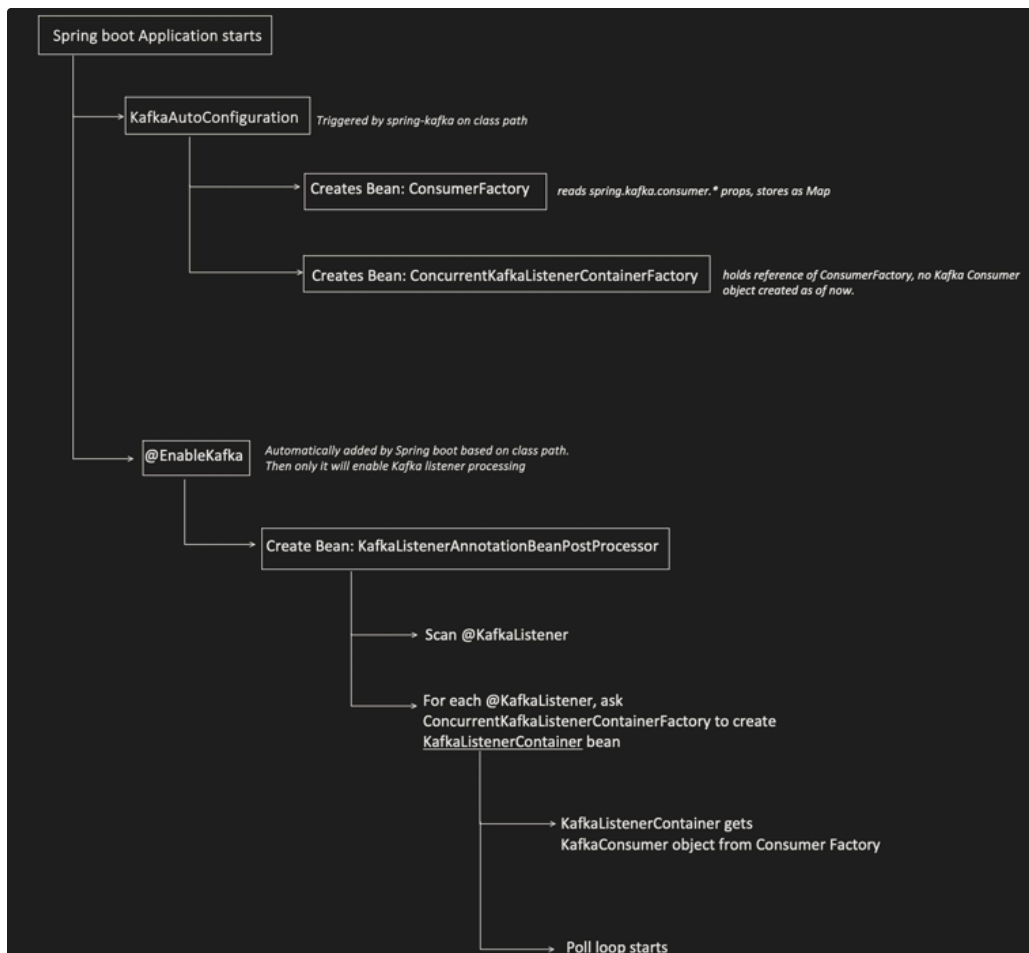
62         @ consumer.commitSync(); // blocks until broker
confirms
63         @ consumer.commitAsync(); // fire-and-forget, no
guarantee
64     }
65 }
66 finally {
67     consumer.close(); // leaves consumer group
68 no heartbeat send by consumer background thread to broker, broker
consider it dead and triggers rebalance
69 }
70 }
71

```

Now good thing is, we don't have to write all of this by ourself, its all provided within Spring Kafka framework.



Flow is:



```

Step5: Partition assignment

Broker3 (Group coordinator)
Group: order-consumer-group

Topic: order-events

Consumer1 : handle Partition-0
Consumer2: handle Partiton-1
Consumer3: handle Partition-2

```

Doubt: How partitions are divided among consumers in the group (range, round robin, sticky, cooperative sticky)

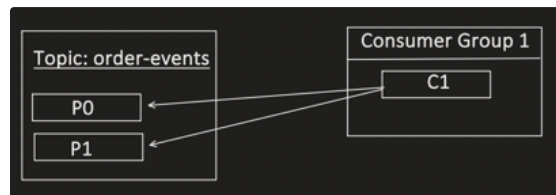
```

1 | spring.kafka.consumer.properties.partition.assignment.strategy

```

Doubt: Till now in above setup, we have created 1 Consumer in a group "order-consumer-group" and which read topic "order-events".

And since "order-events" topic has say 4 partitions, and we have created only 1 consumer in a group, all 4 partitions will be assigned to that 1 consumer **only**.



So how to achieve:

1. **How to do setup of use-case: multiple Consumers in a group reading different partitions of 1 topic?**



- In same application, Concurrency=n

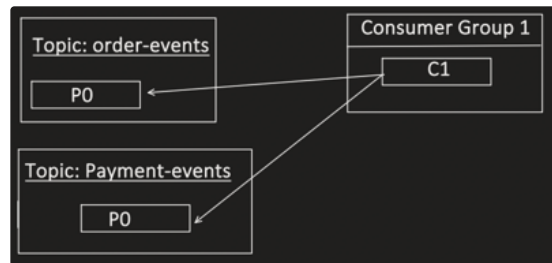
```

1 | #spring.kafka.listener.concurrency=1

```

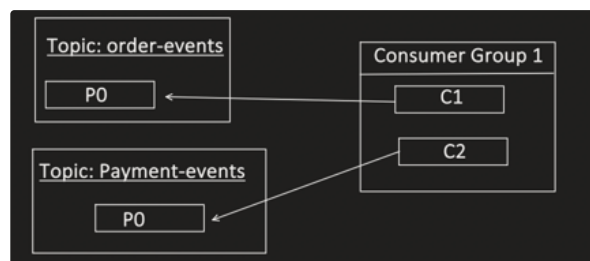
- by creating different consumer application

2. How to do setup of use-case: 1 Consumer reading different partitions of different topic?



- Multiple kafka listener

3. How to do setup of use-case: Multiple Consumer, Multiple topic partition



- By creating different application only.

Step6 and 7:
Consumer assigned P0 of topic
order-events

Consumer ask Group
Coordinator till where the offset
is read for P0 of topic order-
events

Doubt: What if there is no offset present in _consumer_offset topic for topic "order-events" P0?

*If for any topic there is no offset record, then from where read
should start:*

- latest (default, start reading events from end)
- earliest (start reading events from beginning)
- none (no offset found + none, it will throw exception)

Step 8, 9:
Check the metadata and invokes
Leader Broker of
Topic "order-events"
Partition-0

To fetch from given offset
fetched in previous step

Doubt:
- What if there is no event? How much time consumer will wait?

```
spring.kafka.listener.poll.timeout=5000
```

- Is it possible to have gap between polls()

```
spring.kafka.listener.idle.between.polls=0  
  
milliseconds.between.polls@x.x.x.x  
#Default: 0(poll as fast as possible)
```

- How to know, if consumer is stuck with the event processing?

```
spring.kafka.listener.no-poll.threshold=3.0  
- spring.kafka.listener.monitor.interval=30
```

below 3 works together:

```
@spring.kafka.consumer.properties.session.timeout.ms=45000 (heartbeat)  
- spring.kafka.consumer.properties.heartbeat.interval.ms=3000  
- spring.kafka.consumer.properties.max.poll.interval.ms=300000
```

- How consumer will tell, how many events in 1 shot I can get?

```
- spring.kafka.consumer.properties.max.poll.records=500  
- spring.kafka.consumer.properties.fetch.min.bytes=1  
- spring.kafka.consumer.properties.fetch.max.bytes=52428800  
- spring.kafka.consumer.properties.fetch.max.wait.ms=500  
- spring.kafka.consumer.properties.max.partition.fetch.bytes=1048576
```

- What if consumer need to fetch from multiple partition, how its handled. One by one or all in parallel and then club the result and process 1 by 1.

- How long consumer wait for broker to send the response

```
spring.kafka.consumer.properties.request.timeout.ms=30000  
spring.kafka.consumer.properties.connections.max.idle.ms=540000  
spring.kafka.consumer.properties.reconnect.backoff.ms=50  
spring.kafka.consumer.properties.reconnect.backoff.max.ms=1000
```

Step 10:

Consumer process events

- Do my process method, always get 1 record(event) at a time or multiple too?

`spring.kafka.listener.type=single or batch`

Step 11:

Offset commit

Doubt:

- How to manage auto commit? `spring.kafka.consumer.enable-auto-commit`

- How often it does auto commit: `spring.kafka.consumer.properties.auto.commit.interval.ms=5000`

- Manual auto commit strategy

`spring.kafka.listener.ack-mode=batch`

#BATCH(default)—commit after all records from poll() are processed
#RECORD—commit after EACH record is processed
#TIME—commit everyack-time milliseconds
#COUNT—commit everyack-count records
#COUNT_TIME—commit on whichever comes first: count or time
#MANUAL—you call acknowledgment.acknowledge() in your listener
#MANUAL_IMMEDIATE—same as MANUAL but commits immediately (not batched)
#Map to: ContainerProperties.setAckMode()

`spring.kafka.listener.ack-count=10` default is 1

#spring.kafka.listener.ack-time=5000
#Commit after this many milliseconds (only for ack-mode=TIME or COUNT_TIME).
#Default: 5000ms